

Heap 与优先队列

不维护完全有序，怎样持续取得当前极值

408 · 数据结构 Topic DS-05

母问题：为什么只维护局部有序就够了？

堆的生成链是

Priority Workload → Complete-Tree Layout → Root Extremum → Local Repair → Heap Invariant → Cost.

最小堆只保证每个父节点不大于孩子，因此根是全局最小值，但任意两个非祖先节点没有排序承诺。优先队列需要的正是“看极值、删极值、加入新候选”，而不是随时输出完整有序序列。

本册定位与 Owner 边界

- **Owns**：完全二叉树数组布局、父子下标、上滤、下滤、建堆、插入、删除极值与优先队列接口。
- **Uses**：Atlas Foundation 的操作成本向量；DS04 Huffman 和 DS08 图算法只调用优先队列接口。
- **Stops**：一般树遍历归 DS04；heap sort 的整体过程由 DS11 组织；Prim/Dijkstra 的目标不变量归 DS08。

目录

1	表示与核心不变量	2
1.1	堆的三个独立性质	2
1.2	下标体系与手算防错	2
1.3	Sift Down 的选择分支与底向上建堆	2
1.4	堆、优先队列与堆排序的接口分层	3
2	四种状态转移	3
2.1	修改键值与删除任意位置	3
2.2	堆操作的直接推论	3
2.3	双堆维护数据流中位数	3
2.4	合并 K 个有序序列	4
3	优先队列与外部目标	4
4	代码与测试证据	4
5	复杂度、边界与概念分离	6
6	做题控制协议	6

1 表示与核心不变量

完全二叉树按层序放入数组。零基下标下，节点 i 的父节点为 $(i-1)/2$ ，孩子为 $2i+1, 2i+2$ ；越界孩子表示不存在。最小堆不变量为

$$\forall i > 0, \quad A[\text{parent}(i)] \leq A[i].$$

它只承诺根极值；把堆误读成有序数组，是许多“第二小元素/任意两元素比较”错误的来源。

局部有序不等于全局有序

数组 $[2, 5, 3, 9, 7, 8]$ 是最小堆，但 5 与 3 的相对位置没有排序意义。取最小值只看根，删除根后把末元素搬到根并下滤，结构仍保持完全树形。

1.1 堆的三个独立性质

一个数组是否构成堆，必须分别检查三件事：

1. **形状性质**：节点按层序紧凑存储，不能在有效节点之间出现空洞；
2. **序关系性质**：最小堆每条父子边满足 $\text{parent} \leq \text{child}$ ，最大堆方向相反；
3. **数组解释性质**：题目给出的下标起点与孩子公式一致。零基和一基下标的父子公式不能混用。

形状合法而序关系不合法的是“完全二叉树但不是堆”；序关系局部满足但数组中间有空槽的对象也不能直接当作堆。判断一个数组是否为最小堆只需检查所有非叶节点到其存在孩子的边，复杂度为 $O(n)$ ，不需要把数组完整排序。

1.2 下标体系与手算防错

一基数组通常把 $A[0]$ 留空，节点 i 的父节点为 $\lfloor i/2 \rfloor$ ，左右孩子为 $2i, 2i+1$ ；零基数组则使用 $(i-1)/2, 2i+1, 2i+2$ 。两种体系的比较、最后一个非叶节点和建堆起点都随之改变。选择题出现“最后一个非叶节点”时，先确认有效元素个数 n 与下标起点，再写出公式；不要把一基的 $\lfloor n/2 \rfloor$ 与零基数组的代码混在同一行。

检查堆时只需检查父子边，最多检查 $n-1$ 条边；但若题目给的是指针结构，还要先确认完全二叉树形状，不能只看数值关系。最小堆中根是最小值，最大堆中根是最大值；第二小值必在根的两个孩子中，第二大同理，其他排名不能仅凭局部关系确定。

1.3 Sift Down 的选择分支与底向上建堆

根被末元素替换后，若左右孩子都存在，必须先选出更小的孩子，再比较父节点与该孩子；若选错孩子，当前边满足而另一条边可能仍违反堆序。下滤不变量是：待修复元素以上的路径已经合法，待修复元素以下的子树原本合法；每次与更小孩子交换后，问题向更深一层移动，最终到达叶或父子关系恢复。

底向上建堆从最后一个非叶节点倒序下滤。靠近叶子的节点高度小，靠近根的节点数量少，因此总代价为

$$\sum_{h \geq 0} \frac{n}{2^{h+1}} O(h) = O(n).$$

“每个节点最多下滤 $\log n$ 层”只能给出宽松的 $O(n \log n)$ 上界，不能证明紧界；逐个插入确实可能达到 $O(n \log n)$ ，两种建堆过程要分开记账。

若根的两个孩子相等，任取一个都不破坏堆序，但若题目要求稳定的优先级次序，应在关键字后附原始序号并使用二元比较。下滤的停止条件不是“当前节点比某一个孩子小”，而是“当前节点不大于两个存在的孩子”；只有存在更小孩子时才交换。上滤则只需比较父节点，遇到相等通常停止，以避免无意义交换。

自底向上建堆的线性证明可按节点高度分组：高度为 h 的节点最多约 $n/2^{h+1}$ 个，每个最多移动 h 层，故总和 $\sum_{h \geq 0} (n/2^{h+1})h = O(n)$ 。这个证明同时解释为什么靠近底层的节点很多却几乎不移动，靠近根的节点虽能移动很远但数量极少。

1.4 堆、优先队列与堆排序的接口分层

优先队列只承诺 ‘insert’、‘peek-min/max’、‘delete-min/max’ 等操作，堆是其中一种表示。平衡搜索树、配对堆或有序数组也可实现同一接口，但成本向量和稳定性不同。Huffman、Prim、Dijkstra 依赖的是“取当前最小候选并插入新候选”，而不是某个具体数组类。

堆排序利用同一个堆不变量反复把根极值提交到数组末端。若使用最大堆做升序排序，交换根与当前末尾后缩小有效堆区间，再对新根下滤；已提交后缀不再参与堆调整。堆排序的整体过程属于 DS11，但每一趟的根极值、末尾交换和下滤契约由本册提供。堆排序不稳定，且原地性来自在同一数组中缩小有效区，不是来自“堆”三个字自动保证。

2 四种状态转移

插入先把新元素放在数组末尾，再与父节点比较并上滤；每次交换都使元素向根移动一层。删除极值先保存根，把末元素搬到根并缩短长度，再与较小孩子交换并下滤。两种修复都只沿一条根叶路径进行，路径长度为 $h = \Theta(\log n)$ 。

建堆不应逐个调用插入来默认获得最优界。对最后一个非叶节点向前执行下滤，每个子树的高度不同，总成本为 $O(n)$ ；逐个插入则为 $O(n \log n)$ 。这两个结论的前提不同，不能只背“堆是 $\log n$ ”。

2.1 修改键值与删除任意位置

若位置 i 的关键字变小（最小堆），只可能破坏与父节点的关系，沿父链上滤；若关键字变大，只可能破坏与孩子的关系，沿较小孩子下滤。未知修改方向时可先与父节点比较：小于父则上滤，否则从当前位置下滤。删除任意位置时，把末元素搬到 i ，长度减一，再按同样的方向判断修复；若记录每个元素的句柄或数组位置，定位可为 $O(1)$ ，否则先查找位置需 $O(n)$ 。

2.2 堆操作的直接推论

要取前 k 小，可维护大小为 k 的最大堆：扫描每个元素，若大于堆顶则丢弃，否则替换堆顶并下滤，复杂度 $O(n \log k)$ 、额外空间 $O(k)$ 。要取前 k 大则对称使用最小堆。若只需判断一个数组是否为堆，不必建堆；若要把任意数组原地变成堆，采用自底向上下滤。若要连续输出有序序列，堆排序每次提交根后缩短有效区间，但不应因此声称堆本身已完全有序。

2.3 双堆维护数据流中位数

把已经到达的数据拆成左半部最大堆 L 与右半部最小堆 R ，维护

$$\max(L) \leq \min(R), \quad ||L| - |R|| \leq 1.$$

第一条是次序不变量，第二条是规模不变量。插入新值时可先按边界根决定进入哪一侧，再把较大一侧的根移动到另一侧完成再平衡；也可统一“先入 L 、把 L 根移入 R 、再按规模移回”，但每一步都必须恢复两条不变量。奇数个元素时中位数是较大堆的根，偶数时由两个根共同决定；若数值类型可能溢出，求平均前应先扩展类型或改写公式。每次插入 $O(\log n)$ 、查询 $O(1)$ 、空间 $O(n)$ 。

双堆只回答动态秩统计中的中间位置，不自动支持删除任意旧元素。滑动窗口中位数还需位置索引或延迟删除表，并在读取堆顶前清除失效元素；这已经是 **Heap** 与 **Hash** 的组合设计，完整 **workload** 取舍归 DS-I01。

2.4 合并 K 个有序序列

对每条非空序列只把当前首元素放入最小堆，堆项同时携带来源编号和该序列内位置。弹出全局最小项后，只推进同一来源一次，并把它的新首元素重新入堆。循环不变量是：**堆中恰有每条尚未耗尽序列的唯一当前候选，所有已输出元素不大于堆中任何候选**。若总元素数为 N ，堆规模至多 K ，时间为 $O(N \log K)$ ，额外候选空间为 $O(K)$ 。

把所有 N 个元素一次性入堆虽也能排序，却丢失各序列已有序这一信息，成本变为 $O(N \log N)$ 。链表合并时堆中保存节点指针，输出后推进 `next`；外部多路归并时同一接口对应“补入该 **run** 下一条记录”，缓冲与块 I/O 由 DS12 Own。

3 优先队列与外部目标

优先队列抽象为 `push`、`top`、`pop`、`empty`。**Huffman** 需要反复取两个最小权值，**Dijkstra/Prim** 需要取当前最小候选；它们不应复制下滤代码。若优先级相等，是否稳定需要额外的序号字段，堆本身不自动提供稳定性。

4 代码与测试证据

完整实现位于 `code/ds05_heap.hpp`，测试位于 `code/ds05_heap_test.cpp`。代码把“末尾插入—上滤”和“根替换—下滤”分成两个可检查的状态转移。

```

1      build();
2  }
3
4  bool empty() const { return data_.empty(); }
5  std::size_t size() const { return data_.size(); }
6  int top() const {
7      if (empty()) throw std::out_of_range("top on empty heap");
8      return data_.front();
9  }
10 void push(int value) {
11     data_.push_back(value);
12     sift_up(data_.size() - 1);
13 }
14 int pop() {
15     const int result = top();
16     data_.front() = data_.back();
17     data_.pop_back();
18     if (!empty()) sift_down(0);

```

```

19     return result;
20 }
21 bool invariant() const {
22     for (std::size_t i = 1; i < data_.size(); ++i) {
23         if (data_[parent(i)] > data_[i]) return false;
24     }
25     return true;
26 }
27
28 private:
29     std::vector<int> data_;
30     static std::size_t parent(std::size_t i) { return (i - 1) / 2; }
31     static std::size_t left(std::size_t i) { return 2 * i + 1; }
32
33     void sift_up(std::size_t i) {
34         while (i > 0 && data_[parent(i)] > data_[i]) {
35             std::swap(data_[parent(i)], data_[i]);
36             i = parent(i);
37         }
38     }
39     void sift_down(std::size_t i) {
40         while (left(i) < data_.size()) {
41             std::size_t child = left(i);
42             if (child + 1 < data_.size() && data_[child + 1] < data_[child]) ++child;
43             if (data_[i] <= data_[child]) break;
44             std::swap(data_[i], data_[child]);
45             i = child;
46         }
47     }
48     void build() {
49         if (data_.empty()) return;
50         for (std::size_t i = data_.size() / 2; i-- > 0;) sift_down(i);
51     }
52 };
53
54 } // namespace ipara::ds05
55
56 #endif

```

代码清单 1: 堆布局、上滤、下滤与极值接口

```

1
2 MinHeap heap({7, 2, 9, 1, 2});
3 assert(heap.invariant());
4 heap.push(0);
5 assert(heap.top() == 0 && heap.invariant());
6 assert(heap.pop() == 0);
7 assert(heap.pop() == 1);
8 heap.push(3);
9 assert(heap.invariant());
10 assert(heap.pop() == 2);

```

```
11  assert(heap.pop() == 2);
12  assert(heap.pop() == 3);
13  assert(heap.pop() == 7);
14  assert(heap.pop() == 9);
15  assert(heap.empty());
16 }
```

代码清单 2: 空堆、单元素与重复优先级测试

```
1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/ds05_heap_test.cpp -o
   /tmp/ds05_test
2 /tmp/ds05_test
```

5 复杂度、边界与概念分离

操作	时间	额外空间	不变量动作	边界
top	$O(1)$	$O(1)$	只读根	空堆不可取值
push/pop	$O(\log n)$	$O(1)$	上滤/下滤	单元素删除
建堆	$O(n)$	$O(1)$	自底向上	空输入

概念 A	≠ 概念 B	判据	错因
堆序	≠ 完全有序	只比较父子	错误推出任意排名
建堆	≠ 逐个插入	自底向上 vs 重复上滤	误报复杂度
优先队列	≠ 堆实现	接口 vs 具体表示	把调用方绑死

6 做题控制协议

先问目标是否只需当前极值；若是，选择优先队列。再画完全树索引，插入看“末尾后上滤”，删除看“末尾补根后下滤”。每次交换后检查父子关系和数组边界；若题目需要稳定次序、任意第 k 小或完整排序，停止把堆当万能索引，转交 DS09/DS11。

压缩复原

忘记代码时复原五问：根承诺什么？新元素从哪里进入？删除后谁补根？冲突孩子选哪一个？修复结束条件是什么？答案分别恢复根极值、数组末尾、末元素、较小孩子和父子关系成立。